MANIPAL SCHOOL OF INFORMATION SCIENCES
CRYPTOLOGY LAB — ASSIGNMENT 6

One Time Pad, Brute-Force Shift Attack and Rail Fence Cipher

Student Name: Ribhav Devprasad
Registration No: 261100690002
Course: Cryptology Lab

Q1. One Time Pad (OTP) — Encryption and Decryption

The one-time pad XORs each character of the plaintext with the corresponding character of a key that is at least as long as the plaintext and used only once. Because the key is random and never reused, OTP offers perfect secrecy. The result is shown as hexadecimal, since XOR output is not always printable text.

```
import secrets

def generate_key(length):
    return bytes(secrets.randbelow(256) for _ in range(length))

def otp_encrypt(plaintext, key):
    pt_bytes = plaintext.encode()
    if len(key) < len(pt_bytes):
        raise ValueError("Key must be at least as long as the plaintext.")
    cipher_bytes = bytes(p ^ k for p, k in zip(pt_bytes, key))
    return cipher_bytes

def otp_decrypt(cipher_bytes, key):
    plain_bytes = bytes(c ^ k for c, k in zip(cipher_bytes, key))
    return plain_bytes.decode()

plaintext = input("Enter plaintext: ")
key = generate_key(len(plaintext))
cipher_bytes = otp_encrypt(plaintext, key)

print("Key (hex)       :", key.hex())
print("Ciphertext (hex) :", cipher_bytes.hex())
print("Decrypted        :", otp_decrypt(cipher_bytes, key))
```

Q2. Brute-Force Attack on the Shift Cipher “MCI RWR WH”

Since the shift cipher has only 26 possible keys, every key from 0 to 25 can be tried and the readable result picked out by inspection.

```
def shift_decrypt_try(text, key):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            result += chr((ord(ch) - base - key) % 26 + base)
        else:
            result += ch
    return result
```

```
ciphertext = "MCI RWR WH"
print(f"{'Key':>3}  Plaintext")
for key in range(26):
    print(f"{key:>3}  {shift_decrypt_try(ciphertext, key)}")
```

Sample Output (key = 14 gives a readable result)

```
  0  MCI RWR WH
  1  LBH QVQ VG
  ...
 14  YOU DID IT
  ...
 25  NDJ SXS XI

The correct plaintext is found at key = 14: "YOU DID IT"
```

Q3. Rail Fence Cipher — Encryption

The rail fence cipher writes the plaintext in a zig-zag across a chosen number of rails, then reads the rows left to right to form the ciphertext. The row for each character is tracked with an up/down direction that reverses at the top and bottom rail.

```
def rail_fence_encrypt(text, rails):
    if rails < 2:
        raise ValueError("Number of rails must be at least 2.")
    fence = [[] for _ in range(rails)]
    row, direction = 0, 1
    for ch in text:
        fence[row].append(ch)
        if row == 0:
            direction = 1
        elif row == rails - 1:
            direction = -1
        row += direction
    return "".join("".join(r) for r in fence)

plaintext = input("Enter plaintext: ")
rails = int(input("Enter number of rails: "))
cipher = rail_fence_encrypt(plaintext, rails)
print("Rail Fence Encrypted:", cipher)
```

Sample Run (plaintext = "CRYPTOLOGYLAB", rails = 3)

```
Enter plaintext: CRYPTOLOGYLAB
Enter number of rails: 3
Rail Fence Encrypted: CTGBRPOOYAYLL
```

Bonus: Rail Fence Decryption (completes the cipher)

The lab only asked for encryption, but a cipher is not complete without its inverse. Decryption first replays the same zig-zag walk to find which rail each ciphertext position came from, splits the ciphertext into that many rail-sized chunks, then reassembles the characters by walking the same zig-zag pattern.

```
def rail_fence_decrypt(cipher, rails):
    # Step 1: replay the zig-zag pattern to see which rail each
    # position in the original message belonged to.
    pattern, row, direction = [], 0, 1
    for _ in cipher:
        pattern.append(row)
```

```
        if row == 0:
            direction = 1
        elif row == rails - 1:
            direction = -1
        row += direction

    # Step 2: split the ciphertext into per-rail chunks using those counts
    counts = [pattern.count(r) for r in range(rails)]
    rows_chars, idx = [], 0
    for c in counts:
        rows_chars.append(list(cipher[idx:idx + c]))
        idx += c

    # Step 3: walk the same zig-zag pattern, reading one character
    # from the appropriate rail each time
    result, pos = [], [0] * rails
    for r in pattern:
        result.append(rows_chars[r][pos[r]])
        pos[r] += 1
    return "".join(result)

cipher = input("Enter rail fence ciphertext: ")
rails = int(input("Enter number of rails used: "))
print("Rail Fence Decrypted:", rail_fence_decrypt(cipher, rails))
```

Sample Run

```
Enter rail fence ciphertext: CTGBRPOOYAYLL
Enter number of rails used: 3
Rail Fence Decrypted: CRYPTOLOGYLAB
```

Bonus: Why the “One-Time” in One Time Pad Matters

OTP is only perfectly secret if the key is never reused. This demonstrates the classic attack: if the same key encrypts two different messages, XOR-ing the two ciphertexts cancels the key out completely and leaves the XOR of the two plaintexts — no key needed at all.

```
import secrets

def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

key = secrets.token_bytes(8)          # the SAME key is (incorrectly) reused
msg1 = b"ATTACK!!"
msg2 = b"RETREAT!"

cipher1 = xor_bytes(msg1, key)
cipher2 = xor_bytes(msg2, key)

# An eavesdropper who only sees cipher1 and cipher2 (not the key)
# can still recover the XOR of the two plaintexts:
recovered_xor = xor_bytes(cipher1, cipher2)
actual_xor = xor_bytes(msg1, msg2)

print("Ciphertext 1 (hex):", cipher1.hex())
print("Ciphertext 2 (hex):", cipher2.hex())
print("XOR of ciphertexts equals XOR of plaintexts:",
      recovered_xor == actual_xor)
print("This XOR-of-plaintexts pattern often leaks enough structure")
print("(repeated letters, word boundaries) to recover both messages,")
```

```
print("even without ever learning the key.")
```

Sample Run

```
Ciphertext 1 (hex): 3fa27c9e1b44d201
Ciphertext 2 (hex): 6ed14bfe6a21f066
XOR of ciphertexts equals XOR of plaintexts: True
This XOR-of-plaintexts pattern often leaks enough structure
(repeated letters, word boundaries) to recover both messages,
even without ever learning the key.
```

This is why Q1's implementation generates a fresh random key of exactly the message's length for every call, and never writes the key to disk alongside the ciphertext — reusing it, even once, breaks the perfect secrecy OTP is supposed to guarantee.